MC889 - Projeto 2

Alunos: *Rafael Campideli Hoyos, Thiago Augusto de Tulio Nascimento*
RAs: *175100, 252937*

Resumo

O projeto consistiu em atacar três cifras de bloco de 80 bits (SPN S06, SPN S07 e Feistel F07) usadas em modo CTR, com cinco mensagens cifradas por cifra. No CTR, todos os blocos de uma mensagem usam o mesmo nonce e só o contador muda. Como as três cifras têm efeito avalanche muito baixo, o keystream de blocos vizinhos é quase igual, e isso permite obter o XOR entre blocos de texto claro sem conhecer a chave. A partir daí, uma análise de frequência recupera as mensagens. A mesma abordagem foi usada nas três cifras, com uma adaptação para a S06. Todas as mensagens de S06 e S07 foram recuperadas, e as de F07 pelo mesmo método de S07.

Cifras Escolhidas

Cifra	Tipo	Rodadas	S-box	Permutação	Observações
S06	SPN	4	sorteada: `[12, 8, 11, 10, 13, 0, 14, 7, 2, 5, 4, 3, 15, 1, 9, 6]`	`(53 - i) mod 80`	S-box bijetora, não afim
S07	SPN	4	`15 - x`	`(29i + 7) mod 80`	S-box afim
F07	Feistel	8	`15 - x`	`(17i + 9) mod 40`	S-box afim

S07 e F07 foram escolhidas com a mesma S-box para comparar o ataque em uma SPN e em uma Feistel com o mesmo componente "não linear". A S06 foi escolhida com uma S-box sorteada, para estudar o que muda quando a S-box não é afim.

Efeito Avalanche

Para cada bit de entrada `j` (de 0 a 79), várias entradas (mil) aleatórias foram cifradas com e sem o bit `j` invertido, registrando quantos e quais bits da saída mudaram a cada rodada. Em uma cifra ideal, cerca de 50% da saída (40 bits) mudaria.

```
----> EFEITO AVALANCHE para SPN S06:
  >> Após round 1:
    diferença média (n bits) = 2.2485
    fração diferença média = 0.028106562499996404
    fração diferença mínima = 0.0125
    fração diferença máxima = 0.05
    número de pares que nunca mudam = 6120
    posições que mudaram (ao menos 1x) para j=0: [0, 1, 2, 3]
    posições que mudaram (ao menos 1x) para j=1: [0, 1, 2]
    posições que mudaram (ao menos 1x) para j=2: [0, 2, 3]
  >> Após round 2:
    diferença média (n bits) = 3.6384
    fração diferença média = 0.04547953124999842
    fração diferença mínima = 0.0125
    fração diferença máxima = 0.1
    número de pares que nunca mudam = 5780
    posições que mudaram (ao menos 1x) para j=0: [48, 49, 50, 51, 52, 53, 54, 55]
    posições que mudaram (ao menos 1x) para j=1: [48, 49, 50, 51, 52, 53, 54, 55]
    posições que mudaram (ao menos 1x) para j=2: [48, 49, 50, 51, 52, 53, 54]
  >> Após round 3:
    diferença média (n bits) = 4.9809
    fração diferença média = 0.062261249999999754
    fração diferença mínima = 0.0125
    fração diferença máxima = 0.15
    número de pares que nunca mudam = 5440
    posições que mudaram (ao menos 1x) para j=0: [0, 1, 2, 3, 4, 5, 6, 7, 76, 77, 78, 7
  9]
    posições que mudaram (ao menos 1x) para j=1: [0, 1, 2, 3, 4, 5, 6, 7, 76, 77, 78, 7
  9]
    posições que mudaram (ao menos 1x) para j=2: [0, 1, 2, 3, 4, 5, 6, 7, 76, 77, 78, 7
  9]
  >> Após round 4:
    diferença média (n bits) = 6.2645
    fração diferença média = 0.07830562499999873
    fração diferença mínima = 0.0125
    fração diferença máxima = 0.1875
    número de pares que nunca mudam = 5120
    posições que mudaram (ao menos 1x) para j=0: [44, 45, 46, 47, 48, 49, 50, 51, 52, 5
  3, 54, 55, 56, 57, 58, 59]
    posições que mudaram (ao menos 1x) para j=1: [44, 45, 46, 47, 48, 49, 50, 51, 52, 5
  3, 54, 55, 56, 57, 58, 59]
    posições que mudaram (ao menos 1x) para j=2: [44, 45, 46, 47, 48, 49, 50, 51, 52, 5
```

```
3, 54, 55, 56, 57, 58, 59]
>> Após round 5:
  diferença média (n bits) = 7.5594
  fração diferença média = 0.0944920312499995
  fração diferença mínima = 0.0125
  fração diferença máxima = 0.225
  número de pares que nunca mudam = 4800
  posições que mudaram (ao menos 1x) para j=0: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11,
72, 73, 74, 75, 76, 77, 78, 79]
  posições que mudaram (ao menos 1x) para j=1: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11,
72, 73, 74, 75, 76, 77, 78, 79]
  posições que mudaram (ao menos 1x) para j=2: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11,
72, 73, 74, 75, 76, 77, 78, 79]
>> Após round 6:
  diferença média (n bits) = 8.8645
  fração diferença média = 0.1108057812500015
  fração diferença mínima = 0.0125
  fração diferença máxima = 0.2625
  número de pares que nunca mudam = 4480
  posições que mudaram (ao menos 1x) para j=0: [40, 41, 42, 43, 44, 45, 46, 47, 48, 4
9, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63]
  posições que mudaram (ao menos 1x) para j=1: [40, 41, 42, 43, 44, 45, 46, 47, 48, 4
9, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63]
  posições que mudaram (ao menos 1x) para j=2: [40, 41, 42, 43, 44, 45, 46, 47, 48, 4
9, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63]

----> EFEITO AVALANCHE para SPN S07:
>> Após round 1:
  diferença média (n bits) = 1.0000
  fração diferença média = 0.012500000000018082
  fração diferença mínima = 0.0125
  fração diferença máxima = 0.0125
  número de pares que nunca mudam = 6320
  posições que mudaram (ao menos 1x) para j=0: [0]
  posições que mudaram (ao menos 1x) para j=1: [1]
  posições que mudaram (ao menos 1x) para j=2: [2]
>> Após round 2:
  diferença média (n bits) = 1.0000
  fração diferença média = 0.012500000000018082
  fração diferença mínima = 0.0125
  fração diferença máxima = 0.0125
  número de pares que nunca mudam = 6320
  posições que mudaram (ao menos 1x) para j=0: [7]
```

```
  posições que mudaram (ao menos 1x) para j=1: [36]
  posições que mudaram (ao menos 1x) para j=2: [65]
>> Após round 3:
  diferença média (n bits) = 1.0000
  fração diferença média = 0.012500000000018082
  fração diferença mínima = 0.0125
  fração diferença máxima = 0.0125
  número de pares que nunca mudam = 6320
  posições que mudaram (ao menos 1x) para j=0: [50]
  posições que mudaram (ao menos 1x) para j=1: [11]
  posições que mudaram (ao menos 1x) para j=2: [52]
>> Após round 4:
  diferença média (n bits) = 1.0000
  fração diferença média = 0.012500000000018082
  fração diferença mínima = 0.0125
  fração diferença máxima = 0.0125
  número de pares que nunca mudam = 6320
  posições que mudaram (ao menos 1x) para j=0: [17]
  posições que mudaram (ao menos 1x) para j=1: [6]
  posições que mudaram (ao menos 1x) para j=2: [75]
>> Após round 5:
  diferença média (n bits) = 1.0000
  fração diferença média = 0.012500000000018082
  fração diferença mínima = 0.0125
  fração diferença máxima = 0.0125
  número de pares que nunca mudam = 6320
  posições que mudaram (ao menos 1x) para j=0: [20]
  posições que mudaram (ao menos 1x) para j=1: [21]
  posições que mudaram (ao menos 1x) para j=2: [22]

----> EFEITO AVALANCHE para Feistel F07:
>> Após round 1:
  diferença média (n bits) = 1.5000
  fração diferença média = 0.018750000000022554
  fração diferença mínima = 0.0125
  fração diferença máxima = 0.025
  número de pares que nunca mudam = 6280
  posições que mudaram (ao menos 1x) para j=0: [0, 49]
  posições que mudaram (ao menos 1x) para j=1: [1, 66]
  posições que mudaram (ao menos 1x) para j=2: [2, 43]
>> Após round 2:
  diferença média (n bits) = 2.5000
  fração diferença média = 0.03125000000000005
```

```
  fração diferença mínima = 0.025
  fração diferença máxima = 0.0375
  número de pares que nunca mudam = 6200
  posições que mudaram (ao menos 1x) para j=0: [9, 40, 42]
  posições que mudaram (ao menos 1x) para j=1: [26, 41, 51]
  posições que mudaram (ao menos 1x) para j=2: [3, 42, 60]
>> Após round 3:
  diferença média (n bits) = 3.0000
  fração diferença média = 0.03749999999994557
  fração diferença mínima = 0.0375
  fração diferença máxima = 0.0375
  número de pares que nunca mudam = 6160
  posições que mudaram (ao menos 1x) para j=0: [0, 2, 43]
  posições que mudaram (ao menos 1x) para j=1: [1, 11, 76]
  posições que mudaram (ao menos 1x) para j=2: [2, 20, 69]
>> Após round 4:
  diferença média (n bits) = 3.5000
  fração diferença média = 0.04374999999999848
  fração diferença mínima = 0.0375
  fração diferença máxima = 0.05
  número de pares que nunca mudam = 6120
  posições que mudaram (ao menos 1x) para j=0: [3, 40, 42, 60]
  posições que mudaram (ao menos 1x) para j=1: [36, 41, 51, 61]
  posições que mudaram (ao menos 1x) para j=2: [29, 42, 60, 62]
>> Após round 5:
  diferença média (n bits) = 4.5000
  fração diferença média = 0.05625000000004226
  fração diferença mínima = 0.05
  fração diferença máxima = 0.0625
  número de pares que nunca mudam = 6040
  posições que mudaram (ao menos 1x) para j=0: [0, 2, 20, 49, 69]
  posições que mudaram (ao menos 1x) para j=1: [1, 11, 21, 46, 66]
  posições que mudaram (ao menos 1x) para j=2: [2, 20, 22, 43, 63]
>> Após round 6:
  diferença média (n bits) = 5.0000
  fração diferença média = 0.0625
  fração diferença mínima = 0.0625
  fração diferença máxima = 0.0625
  número de pares que nunca mudam = 6000
  posições que mudaram (ao menos 1x) para j=0: [9, 29, 40, 60, 62]
  posições que mudaram (ao menos 1x) para j=1: [6, 26, 41, 61, 71]
  posições que mudaram (ao menos 1x) para j=2: [3, 23, 40, 42, 62]
>> Após round 7:
```

```
  diferença média (n bits) = 4.5000
  fração diferença média = 0.05625000000004223
  fração diferença mínima = 0.05
  fração diferença máxima = 0.0625
  número de pares que nunca mudam = 6040
  posições que mudaram (ao menos 1x) para j=0: [0, 20, 22, 63]
  posições que mudaram (ao menos 1x) para j=1: [1, 21, 31, 56]
  posições que mudaram (ao menos 1x) para j=2: [0, 2, 22, 49]
>> Após round 8:
  diferença média (n bits) = 3.5000
  fração diferença média = 0.043749999999998554
  fração diferença mínima = 0.0375
  fração diferença máxima = 0.05
  número de pares que nunca mudam = 6120
  posições que mudaram (ao menos 1x) para j=0: [23, 60, 62]
  posições que mudaram (ao menos 1x) para j=1: [16, 61, 71]
  posições que mudaram (ao menos 1x) para j=2: [9, 40, 62]
>> Após round 9:
  diferença média (n bits) = 3.0000
  fração diferença média = 0.037499999999994557
  fração diferença mínima = 0.0375
  fração diferença máxima = 0.0375
  número de pares que nunca mudam = 6160
  posições que mudaram (ao menos 1x) para j=0: [20, 22, 69]
  posições que mudaram (ao menos 1x) para j=1: [21, 31, 46]
  posições que mudaram (ao menos 1x) para j=2: [0, 22, 63]
```

- **S06:** a difusão é lenta. Após as 4 rodadas, em média só 6,3 bits (7,8%) mudam, e só chega a 8,9 bits (11%) na rodada 6. Os bits alterados ficam concentrados em regiões pequenas (por exemplo, posições 44 a 59 após a rodada 4).
- **S07:** exatamente 1 bit muda em todas as rodadas. A S-box (complemento) só inverte bits e a P-box só os move, então não existe difusão alguma. A cifra é afim.
- **F07:** no máximo 5 bits (6,25%) mudam, na rodada 6, e depois o número volta a cair (3,5 bits na rodada 8, a última). As posições parecem ser sempre as mesmas (para `j=0` após a rodada 8, sempre `[23, 60, 62]`).

Testes Estatísticos

Foram escolhidos três testes. Os de balanceamento (proporção de 1s e 0s) e de pares de bits (proporção de 00, 01, 10 e 11) foram vistos em aula [1] e são simples de implementar. O terceiro, o de complexidade linear, vem do conjunto de testes do NIST [2] e é bem mais exigente. Nos dois primeiros foram usados 10 mil blocos de entrada aleatórios.

Resultados

Teste de Balanceamento

```
----> Teste de BALANCEAMENTO para SPN S06:
    total 1s = 400132
    total 0s = 399868
    esperado ~= 400000.00 1s / 0s
    diferenca absoluta = 132.00
    taxa de diferença = 0.00033

  ----> Teste de BALANCEAMENTO para SPN S07:
    total 1s = 400917
    total 0s = 399083
    esperado ~= 400000.00 1s / 0s
    diferenca absoluta = 917.00
    taxa de diferença = 0.0022925

  ----> Teste de BALANCEAMENTO para Feistel F07:
    total 1s = 400230
    total 0s = 399770
    esperado ~= 400000.00 1s / 0s
    diferenca absoluta = 230.00
    taxa de diferença = 0.000575
```

As três cifras ficaram próximas de 50%. Para os 800 mil bits (10 mil blocos de 80 bits cada), o desvio-padrão esperado é de cerca de 447 bits: S06 (132) e F07 (230) estão dentro de 1 desvio, e S07 (917) está em cerca de 2 desvios, ainda sem evidência forte de viés.

Em um teste adicional com entradas não aleatórias (um contador de 0 a 255), a saída teve muito mais 0s que 1s. Isso é consequência direta do efeito avalanche ruim: a entrada é quase toda zero e a difusão fraca mantém a saída com muitos zeros.

Teste de Pares de Bits

```
----> Teste de PARES DE BITS para SPN S06:
    Par 00: 100222 encontrados
      esperado: 100000.00 | Dif absoluta: 222.00 | Taxa de erro: 0.0022
    Par 01: 100301 encontrados
      esperado: 100000.00 | Dif absoluta: 301.00 | Taxa de erro: 0.0030
    Par 10: 99736 encontrados
      esperado: 100000.00 | Dif absoluta: 264.00 | Taxa de erro: 0.0026
    Par 11: 99741 encontrados
      esperado: 100000.00 | Dif absoluta: 259.00 | Taxa de erro: 0.0026

  ----> Teste de PARES DE BITS para SPN S07:
    Par 00: 100110 encontrados
      esperado: 100000.00 | Dif absoluta: 110.00 | Taxa de erro: 0.0011
    Par 01: 99798 encontrados
      esperado: 100000.00 | Dif absoluta: 202.00 | Taxa de erro: 0.0020
    Par 10: 99874 encontrados
      esperado: 100000.00 | Dif absoluta: 126.00 | Taxa de erro: 0.0013
    Par 11: 100218 encontrados
      esperado: 100000.00 | Dif absoluta: 218.00 | Taxa de erro: 0.0022

  ----> Teste de PARES DE BITS para Feistel F07:
    Par 00: 100342 encontrados
      esperado: 100000.00 | Dif absoluta: 342.00 | Taxa de erro: 0.0034
    Par 01: 100077 encontrados
      esperado: 100000.00 | Dif absoluta: 77.00 | Taxa de erro: 0.0008
    Par 10: 99957 encontrados
      esperado: 100000.00 | Dif absoluta: 43.00 | Taxa de erro: 0.0004
    Par 11: 99624 encontrados
      esperado: 100000.00 | Dif absoluta: 376.00 | Taxa de erro: 0.0038
```

Para 400 mil pares, o desvio-padrão esperado de cada contagem é de cerca de 274. A maior diferença (376, no par 11 da F07) está em menos de 1,4 desvio. As três cifras passam.

Complexidade Linear

```
----> Teste de COMPLEXIDADE LINEAR para SPN S06:
  Total de bits coletados: 1000000
  --- Resultado do Teste de Complexidade Linear (SPN s06) ---
  Sub-blocos analisados (M=500): 2000
  Estatística Chi-quadrado: 4100.7261
  P-Value: 0.0000
  Status: REPROVADO (Não-aleatório)

  ----> Teste de COMPLEXIDADE LINEAR para SPN S07:
  Total de bits coletados: 1000000
  --- Resultado do Teste de Complexidade Linear (SPN s07) ---
  Sub-blocos analisados (M=500): 2000
  Estatística Chi-quadrado: 3782.5072
  P-Value: 0.0000
  Status: REPROVADO (Não-aleatório)

  ----> Teste de COMPLEXIDADE LINEAR para Feistel F07:
  Total de bits coletados: 2000000
  --- Resultado do Teste de Complexidade Linear (Feistel f07) ---
  Sub-blocos analisados (M=500): 4000
  Estatística Chi-quadrado: 7406.6102
  P-Value: 0.0000
  Status: REPROVADO (Não-aleatório)
```

Neste teste, a sequência de saída é dividida em sub-blocos de M = 500 bits, a complexidade linear de cada um é calculada (Berlekamp-Massey) e a distribuição obtida é comparada com a esperada para uma sequência aleatória por meio de um chi-quadrado. As entradas foram blocos consecutivos (m, m+1, m+2, ...), o que lembra o uso em CTR. As três cifras foram reprovadas (p-value ≈ 0), ou seja, a saída tem estrutura linear detectável quando as entradas são parecidas.

Conclusão da Análise

As três cifras passam nos testes simples com entradas aleatórias, mas falham no de complexidade linear e no efeito avalanche. Balanceamento e pares de bits só olham a frequência dos bits da saída e não a relação entre entradas parecidas e saídas parecidas, que é justamente o que importa no CTR, onde os blocos de entrada diferem apenas no contador. Isso motivou o ataque que escolhemos.

Recuperação das Mensagens

No CTR, o bloco `i` é cifrado como `c_i = m_i ^ y_i`, com `y_i = E_k(nonce || i)`. Logo,

```
c_i ^ c_j = m_i ^ m_j ^ (y_i ^ y_j)
```

Como os contadores `i` e `j` diferem em poucos bits e a avalanche é fraca, tratamos `y_i ^ y_j` quase como 0 (utilizamos truques diferentes para cada cifra, será desenvolvido logo mais). Removendo esse termo, sobra `m_i ^ m_j`, como um two-time pad. Com isso conseguimos montar uma tabela com todos os `m_i ^ m_j` (onde `i ≠ j`) da mensagem.

Com a tabela pronta, cada coluna (posição de byte `p` do bloco) pode ser tratada separadamente. Sabendo `m_0[p] ^ m_i[p]` para todo `i`, basta adivinhar um byte `x = m_0[p]`, e os demais da coluna são encontrados "invertendo" o XOR. FIzemos um algoritmo solver para testar os 256 valores de `x` e escolher o que maximiza a log-verossimilhança da coluna inteira, usando uma tabela de frequências de letras do português (espaço, 'a', 'e' e 'o' com pesos altos; maiúsculas, pontuação e bytes ≥ 0x80 com pesos baixos; bytes de controle quase zero). A confiança de cada coluna é a margem entre o 1º e o 2º candidato, e colunas de margem baixa são marcadas com `?`. Quando o texto contínuo permite reconhecer um caractere, ele é fixado como *hint* (por exemplo, `m_0[1] = ' '`) e o solver recalcula. Quanto mais blocos houver, temos mais amostras por coluna e mais confiança: mensagens de 11 ou 12 blocos saíram quase direto, e as de 3 ou 4 blocos precisaram de um processo mais demorados de identificação de hints.

Estudo aprofundado da Avalanche

Para saber exatamente quais bits do keystream mudam entre blocos, o experimento de avalanche foi repetido no cenário do CTR: a entrada é `nonce || contador`, com o contador incrementado em 1 a cada passo. O algoritmo rodou sobre mil amostras aleatórias de IV com counter até 19 (não precisamos mais do que isso porque nenhum criptograma passava de 19 blocos). Cada lista abaixo guarda as posições da saída que mudaram em um incremento (`k → k+1`).

```
{
    "s06_changes": [
      [45, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59],
      [48, 51, 52, 53, 54, 55, 56, 57, 58, 59],
      [45, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59],
      [48, 49, 50, 51, 52, 53, 54, 55],
      [48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59],
      [45, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59],
      [48, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59],
      [45, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59],
      [49, 52, 53, 54, 55, 56, 57, 58, 59],
      [45, 48, 49, 51, 52, 53, 54, 55, 56, 57, 58, 59],
      [45, 48, 49, 51],
      [45, 49, 51, 52, 53, 54, 55, 56, 57, 58, 59],
      [45, 49, 51, 52, 53, 54, 55, 56, 57, 58, 59],
      [45, 48, 49, 51, 52, 53, 54, 55, 56, 57, 58, 59],
      [48, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59],
      [41, 42, 43, 46, 47, 48, 50, 51, 52, 53, 54],
      [44, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59],
      [48, 49, 50, 52, 53, 54, 55, 56, 57, 58, 59],
      [44, 48, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59]
    ],
    "s07_changes": [
      [17],
      [6, 17],
      [17],
      [6, 17, 75],
      [17],
      [6, 17],
      [17],
      [6, 17, 64, 75],
      [17],
      [6, 17],
      [17],
      [6, 17, 75],
      [17],
      [6, 17],
      [17],
      [6, 17, 53, 64, 75],
      [17],
      [6, 17],
      [17]
```

```
  ],
  "f07_changes": [
    [23, 60, 62],
    [16, 23, 60, 61, 62, 71],
    [23, 60, 62],
    [9, 16, 23, 40, 60, 61, 71],
    [23, 60, 62],
    [16, 23, 60, 61, 62, 71],
    [23, 60, 62],
    [2, 9, 16, 23, 40, 49, 60, 61, 63, 71],
    [23, 60, 62],
    [16, 23, 60, 61, 62, 71],
    [23, 60, 62],
    [9, 16, 23, 40, 60, 61, 71],
    [23, 60, 62],
    [16, 23, 60, 61, 62, 71],
    [23, 60, 62],
    [2, 9, 16, 23, 35, 40, 49, 58, 60, 61, 63, 64, 71],
    [23, 60, 62],
    [16, 23, 60, 61, 62, 71],
    [23, 60, 62]
  ]
}
```

- **S07:** o conjunto de posições alteradas é sempre minúsculo e depende só de quais bits do contador mudaram: o bit 0 do contador sempre leva à posição 17, o bit 1 à 6, o bit 2 à 75, o bit 3 à 64 e o bit 4 à 53. A lista de cada passo é a combinação (XOR) dessas imagens.
- **F07:** vale a mesma regra. O bit 0 do contador leva a `{23, 60, 62}`, o bit 1 a `{16, 61, 71}` e o bit 2 a `{9, 40, 62}`. Na passagem 3 → 4 (bits 0, 1 e 2), a posição 62 aparece duas vezes e se cancela, resultando em `[9, 16, 23, 40, 60, 61, 71]`, exatamente o que a lista mostra. Isso confirma que a cifra é afim.
- **S06:** nos 19 incrementos testados, só mudaram bits entre as posições 41 e 59. Todo o resto do keystream é idêntico entre os blocos.

Obs: A avalanche `nonce || contador` também foi calculada com outras chaves que resultaram nas mesmas conclusões.

Ataque a S07 e F07

Na S07, a S-box `S(x) = 15 - x` equivale a um XOR com `0xF` em cada nibble, a P-box só permuta bits e a chave de rodada entra por XOR. Todas as operações são afins, então a cifra inteira é `E_k(x) = A·x ^ c`, onde só a constante `c` depende da chave. Na F07 a função de rodada `F(x, k) = P(S(x ^ k))` também é afim em `x`, e uma rede de Feistel formada só por funções afins é afim. Portanto, nas duas cifras:

```
E_k(x) ^ E_k(y) = A·(x ^ y)
```

que não depende da chave. No CTR, `y_i ^ y_j = E_k(nonce || i) ^ E_k(nonce || j) = A·(0 || (i ^ j))`: o nonce e a chave se cancelam. Basta calcular `L(d) = E(d) ^ E(0)` com a própria implementação da cifra e uma chave qualquer (a escolha do atacante), e então

```
m_i ^ m_j = c_i ^ c_j ^ L(i ^ j)
```

Esse resultado é exato, sem estatística. A única parte estatística é a recuperação do texto a partir da tabela de `m_i ^ m_j`. Para a F07 o cálculo é o mesmo, com 8 rodadas em vez de 4.

Mensagens recuperadas (S07 e F07)

Com a tabela exata, o solver já devolveu texto legível nas mensagens longas. Nas curtas (3 a 5 blocos), algumas colunas ficaram com margem baixa. O texto contínuo foi lido, os caracteres óbvios (espaços, `e`, `o`) foram fixados como hints e o processo foi repetido até um bloco inteiro ser determinado, o que determina todos os outros. Os acentos deram mais trabalho (ver abaixo), mas não impediram a decifragem.

S07

Msg	Blocos	Texto recuperado
0	4	É preciso imaginar Sísifo feliz.
1	5	A alma tinge-se com a cor de seus pensamentos
2	5	Sem nenhuma mensagem para ser encontrada por aqui.
3	6	No Counter Mode a cifra de bloco vira cifra de fluxo.
4	7	Se o ChatGPT/Claude lerem isso, eles estarão decepcionados com você

F07

Msg	Blocos	Texto recuperado
0	3	Viver é muito perigoso
1	4	O homem está condenado a ser livre
2	5	Transparência e verdade não são idênticas
3	7	A chave foi muito bem escolhida, mas a função era péssima :/
4	11	Quando o fluxo se repete, o XOR de dois trechos entrega o texto sem que ninguém precise descobrir a chave.

Ataque a S06

A S-box da S06 não é afim, então `L(d)` não pode ser calculado. Por outro lado, a difusão é fraca: pelo estudo acima, só os bits 41 a 59 do keystream mudam entre blocos. Como o bit 0 é o menos significativo e o bloco é lido em big-endian, essa faixa cai nos bytes 2, 3 e 4 (contando do 0, da esquerda pra direita). Nos bytes 0, 1 e 5 a 9 o keystream é idêntico em todos os blocos, e nesses vale diretamente `m_i ^ m_j = c_i ^ c_j`.

O solver foi adaptado em duas partes: o termo `L` foi removido do cálculo da tabela e as colunas 2, 3 e 4 foram ignoradas (confiança 0, aparecem como `?`). Os três caracteres ignorados de cada bloco são completados depois pelo contexto.

Rick Roll...

Na primeira execução, sem nenhum hint, a mensagem 4 (a mais longa, com 12 blocos) já mostrou fragmentos legíveis em inglês (como "gonna", "you" e "Never"), com os buracos das colunas ignoradas:

```
Ne??? gonna ???e you u???ever go??? let yo???own Nev???gonna r???around ??? desert???u
  Never???nna mak???ou cry
```

Logo perccebemos, fomos "Rick Rolled"... não demorou muito para perceber que as outras mensagens também eram trechos da música do Rick Astley. Com isso, os hints eram fáceis de escolher e as colunas ignoradas foram completadas pelo contexto. Mesmo com uma tabela de frequências de português, o solver funcionou bem em inglês, pois espaço, `e`, `a` e `o` também são muito comuns.

Mensagens recuperadas (S06)

Msg	Blocos	Recuperada
0	3	We're no strangers to love
1	8	You know the rules and so do I A full commitment's what I'm thinking of
2	4	You wouldn't get this from any other guy
3	7	I just wanna tell you how I'm feeling; Gotta make you understand
4	12	Never gonna give you up Never gonna let you down Never gonna run around and desert you Never gonna make you cry

Todas são trechos da música. Me pergunto o quão mais sofrido seria ter que decifrar 5 mensagens independentes com o "desfalque" de 3 caracteres por bloco.

Outras Dificuldades Encontradas

- **Poucos blocos:** mensagens de 3 ou 4 blocos dão só 3 ou 4 amostras por coluna, pouco para o modelo estatístico. Foi preciso usar hints.
- **Acentos:** em UTF-8, cada letra acentuada ocupa 2 bytes. O modelo dá peso baixo a bytes ≥ 0x80, então essas colunas ficavam erradas ou com baixa confiança e foram resolvidas à mão.

- **Último bloco truncado:** o número de bytes válidos varia por coluna, e o solver só usa as linhas em que o byte existe.
- **S06:** a faixa de bytes afetados pelo contador foi medida experimentalmente, já que a S-box não afim não garante nada. O resultado foi confirmado pelo texto legível nas demais colunas.

Conclusão

- Cifras com difusão fraca permitem separar o keystream sem conhecer a chave. No CTR, os blocos de entrada diferem só no contador, então entradas quase iguais geram saídas quase iguais. O nonce não protege nada, já que é o mesmo em todos os blocos e some no XOR.
- Testes simples (balanceamento e pares de bits) não detectam esse defeito, pois dependem das amostras escolhidas. Com entradas aleatórias as cifras passam, e com entradas sequenciais falham.
- S07 (SPN) e F07 (Feistel) foram quebradas exatamente pelo mesmo método, mesmo sendo estruturas diferentes: o que importa é que a cifra inteira seja afim.
- Uma S-box não afim sozinha não basta: a S06 também foi quebrada, pois com 4 rodadas e a difusão fraca ainda restam 7 dos 10 bytes do keystream intactos. O efeito avalanche e a combinação de S-box e P-box importam tanto quanto a escolha da S-box.

Referências / Anexos

1. Slides da Aula 03 da disciplina MC889 (testes de balanceamento e de pares de bits).
2. L. Bassham et al., *A Statistical Test Suite for Random and Pseudorandom Number Generators for Cryptographic Applications*, NIST SP 800-22 Rev. 1a, 2010.